Programación III
Presentación de Cátedra
Tipos de predicados

Alejandro Sánchez
aljsanchez@gmail.com

Departamento de Informática
Universidad Nacional de San Luis

San Luis

Variedades de Predicados

- Predicados Lógicos Puros
  Relaciones declarativas basadas estrictamente en la lógica de cláusulas de Horn
- Metapredicados
  Predicados que operan sobre otros predicados o metas como datos (Orden Superior)
- Extrapredicados (Extralógicos)
  Predicados que introducen efectos secundarios, interactúan con el entorno o controlan la ejecución

Programación III
Presentación de Cátedra – Tipos de predicados

1. Predicados lógicos puros
2. Metapredicados
3. Extrapredicados

Predicados Lógicos Puros

- Relaciones basadas únicamente en la unificación y en la resolución de cláusulas de Horn
- Sin intervención en el mecanismo de inferencia
- Semántica declarativa pura:
  Representan conocimientos verdaderos o falsos.
- Reversibilidad plena (multidireccionalidad):
  Argumentos como entradas o salidas de forma indistinta .
- Transparencia referencial:
  No modifican el estado ni dependen del entorno.

Ejemplos Clásicos:

- `padre/2`, `ancestro/2`, `append/3`, `member/2`, `length/2`.

Programación III
Presentación de Cátedra – Tipos de predicados

1. Predicados lógicos puros
2. Metapredicados
3. Extrapredicados

Metapredicados

- Predicados que aceptan como argumentos a otros objetivos (metas) o cláusulas para evaluarlos o manipularlos dinámicamente.
- Aprovechan la *homoiconicidad* de Prolog
  - El código y los datos comparten la misma representación como términos.
  - Un objetivo es una estructura de datos ejecutable.
- Permiten implementar *programación de orden superior* (*Higher-Order Programming*).
- Abstraen control de flujo personalizados: patrones de búsqueda, filtrado y agregación.

Ejemplos Clásicos:

- `call/N`, `\+/1` (negación por falla), `findall/3`, `bagof/3`, `setof/3`, `forall/2`, `maplist/N`.

- Evaluación dinámica: `call(Objetivo)`
- Currificación: `call(Clausura, Arg1, ..., ArgN)`

```
es_par(N) :- 0 is N mod 2.
mayor_que(Limite, N) :- N > Limite.

?- Meta = es_par(4), call(Meta).
true.

?- call(mayor_que(5), 8).
true.

mi_include(_,[],[]).
mi_include(Pred,[H|T],[H|Resto]) :-
call(Pred,H), !, mi_include(Pred,T,Resto).
mi_include(Pred,[_|T],Resto) :-
 ↪  mi_include(Pred,T,Resto).
```

Control de Flujo

- Metapredicado predefinido
- Permiten manipular la semántica de demostración y los puntos de elección
  - `once/1`
  - `ignore/1`
  - `forall/2`

- `once/1`

```
% once(Goal): Ejecuta Goal
% y elimina puntos de elección
% Equivalente a: once(G) :- call(G), !.
?- once(member(X, [8, 16, 17])).
X = 8.

?- once(member(X, [])).
false.
```

- `ignore/1`

```
% ignore(Goal): Nunca falla
% Ejecuta Goal; si falla, tiene éxito silenciosamente
% Equivalente a:
% ignore(G) :- G, !.
% ignore(_).
?- ignore(fail).
true.
```

- `forall/2`

```
% forall(Condicion, Accion):
% Cuantificación Universal (P => Q)
% Equivalente declarativo: \+ (Condicion, \+ Accion)
?- forall(member(X, [8, 10, 12]), 0 is X mod 2).
true.
```

Recolección de Soluciones

- Metapredicados predefinidos
- Transforman soluciones encontradas por *backtracking* en una lista explícita en memoria
  - `findall/3`
  - `bagof/3`
  - `setof/3`

- `findall/3`

```
edad(ana, 25).
edad(luis, 30).
edad(maria, 25).

% findall(Template, Goal, List):
% Nunca falla, incluye duplicados
?- findall(E, edad(_, E), Edades).
Edades = [25, 30, 25].
```

- `bagof/3`

```
edad(ana, 25).
edad(luis, 30).
edad(maria, 25).

% bagof(Template, Goal, List):
% Agrupa por variables libres
?- bagof(Nom, edad(Nom, E), Nombres).
E = 25, Nombres = [ana, maria] ;
E = 30, Nombres = [luis].
```

- setof/3

```
edad(ana, 25).
edad(luis, 30).
edad(maria, 25).

% setof(Template, Goal, List):
% Ordena y elimina duplicados
% Nom^edad(Nom, E) le indica que ignore Nom
?- setof(E,Nom^edad(Nom, E), EdadesUnicas).
EdadesUnicas = [25, 30].

?- setof(E,edad(Nom, E), EdadesUnicas).
EdadesUnicas = [25, 30].
Nom = ana,
EdadesUnicas = [25] ;
Nom = luis,
EdadesUnicas = [30] ;
Nom = maria,
EdadesUnicas = [25].
```

- Metapredicados predefinidos
- Garantizan la liberación de recursos en código no determinista o ante excepciones
  - `setup_call_cleanup/3`
  - `catch/3`

- `setup_call_cleanup/3` Asegura que `Cleanup` se ejecute exactamente una vez cuando `Goal` termina (éxito, falla, corte o excepción)

```
% setup_call_clean(Setup, Goal, Cleanup)
% Lectura segura de un archivo
% con limpieza garantizada
leer_primera_linea(Archivo, Linea) :-
  setup_call_cleanup(
    open(Archivo, read, Stream),
    read_line_to_string(Stream, Linea),
    close(Stream)
  ).
```

- `catch/3`: Captura de excepciones

```
?- catch(X is 5 / 0, Error,
↪  format('Error capturado: ~w~n', [Error])).
```

Programación III
Presentación de Cátedra – Tipos de predicados

1. Predicados lógicos puros
2. Metapredicados
3. Extrapredicados

Extrapredicados

Los **extrapredicados** o **predicados extralógicos** son aquellos cuyas operaciones se encuentran **fuera del modelo formal de la lógica de primer orden**.

- Producen **efectos secundarios** (*side effects*) en el entorno o en el estado del intérprete.
- **No son reversibles:** Tienen una dirección de ejecución fija.
- Rompen la semántica declarativa pura (el orden de evaluación importa de forma crítica).
- Permiten interactuar con el usuario, la memoria, archivos y el control del algoritmo.

Clasificación de Extrapredicados

Los extrapredicados se dividen conceptualmente en cuatro subclases principales:

1. **Entrada y Salida (E/S):**
   - `read/1`, `write/1`, `nl/0`, `format/2`, `open/3`, `close/1`.
2. **Manipulación de la Base de Datos Dinámica:**
   - `asserta/1`, `assertz/1`, `retract/1`, `retractall/1`, `abolish/1`.
3. **Inspección Metalógica de Términos:**
   - `var/1`, `nonvar/1`, `atom/1`, `integer/1`, `compound/1`, `functor/3`, `arg/3`, `=../2`.
4. **Control del Intérprete y Excepciones:**
   - `!/0` (corte), `fail/0`, `repeat/0`, `halt/0`, `throw/1`, `catch/3`.

Base de Datos Dinámica

Permiten agregar/eliminar cláusulas durante la ejecución.

```
:- dynamic contador/1.

contador(0).

% Incrementar el contador global
incrementar :-
    retract(contador(N)),
    N1 is N + 1,
    assertz(contador(N1)).

% Consulta:
?- contador(C).
C = 0.
?- incrementar.              % Modifica la BD dinámica
?- contador(C).
C = 1.
```

Uso de `repeat/0`, `read/1`, `write/1` y `!/0` para crear un bucle interactivo:

```
menu_interactivo :-
   repeat,
   write('Ingrese una opcion (salir para terminar): '),
   read(X),
   format('Usted ingreso: ~w~n', [X]),
   X == salir,
   !. % Corte para detener la repeticion por
   ↪  backtracking
```

Atención con `repeat/0`

`repeat/0` siempre tiene éxito y genera infinitos puntos de elección al re-rastrear. Requiere un corte (`!`) posterior cuando se alcanza la condición de parada.